# PyTorch CHEAT SHEET

1 Load data 2 Define model 3 Train model 4 Evaluate model

## General

PyTorch is a open source machine learning framework. It uses **torch.Tensor** – multi-dimensional matrices – to process. A core feature of neural networks in PyTorch is the autograd package, which provides automatic derivative calculations for all operations on tensors.

import torch.optim as — from torch.utils.data import

| | | | |
|---|---|---|---|
| **import torch** | Root package | **torch.randn(*size)** | Create random tensor |
| **import torch.nn as nn** | Neural networks | **torch.Tensor(L)** | Create tensor from list |
| **from torchvision import datasets, models, transforms** | Popular image datasets, architectures & transforms | **tnsr.view(a,b, ...)** | Reshape tensor to size (a, b, ...) |
| **import torch.nn.functional as F** | Collection of layers, activations & more | **requires_grad=True** | tracks computation history for derivative calculations |

## Layers

**nn.Linear(m, n)**: Fully Connected layer (or dense layer) from m to n neurons

**nn.ConvXd(m, n, s)**: X-dimensional convolutional layer from m to n channels with kernel size s; X ∈ {1, 2, 3}

**nn.Flatten()**: Flattens a contiguous range of dimensions into a tensor

**nn.MaxPoolXd(s)**: X-dimensional pooling layer with kernel size s; X ∈ {1, 2, 3}

**nn.Dropout(p=0.5)**: Randomly sets input elements to zero during training to prevent overfitting

**nn.BatchNormXd(n)**: Normalizes a X-dimensional input batch with n features; X ∈ {1, 2, 3}

**nn.Embedding(m, n)**: Lookup table to map dictionary of size m to embedding vector of size n

**nn.RNN/LSTM/GRU**: Recurrent networks connect neurons of one layer with neurons of the same or a previous layer

*torch.nn offers a bunch of other building blocks.*
*A list of state-of-the-art architectures can be found at https://paperswithcode.com/sota.*

## Load data

A dataset is represented by a class that inherits from **Dataset** (resembles a list of tuples of the form (features, label)).

**DataLoader** allows to load a dataset without caring about its structure.

Usually the dataset is split into training (e.g. 80%) and test data (e.g. 20%).

```
from torch.utils.data
 import Dataset, TensorDataset,
        DataLoader, random_split

train_data, test_data =
 random_split(
    TensorDataset(inps, tgts),
    [train_size,test_size]
 )

train_loader =
     DataLoader(
        dataset=train_data,
        batch_size=16,
        shuffle=True)
```

## Activation functions

Common activation functions include **ReLU**, **Sigmoid** and **Tanh**, but there are other activation functions as well.

**nn.ReLU()** *creates a* nn.Module *for example to be used in Sequential models.* **F.relu()** *ist just a call of the ReLU function e.g. to be used in the* forward *method.*

**nn.ReLU()** or **F.relu()**
Output between 0 and ∞,
most frequently used activation function

**nn.Sigmoid()** or **F.sigmoid()**
Output between 0 and 1,
often used for predicting probabilities

**nn.Tanh()** or **F.tanh()**
Output between -1 and 1,
often used for classification with two classes

## Define model

There are several ways to define a neural network in PyTorch, e.g. with **nn.Sequential** (a), as a class (b) or using a combination of both.

```
model = nn.Sequential(
  nn.Conv2D(■,■,■)
  nn.ReLU()
  nn.MaxPool2D(■)
  nn.Flatten()
  nn.Linear(■,■)
)
```
a

```
class Net(nn.Module):
  def __init__():
    super(Net, self).__init__()
    self.conv
      = nn.Conv2D(■,■,■)
    self.pool
      = nn.MaxPool2D(■)
    self.fc = nn.Linear(■,■)

  def forward(self, x):
    x = self.pool(
        F.relu(self.conv(x))
      )
    x = x.view(-1,■)
    x = self.fc(x)
    return x
model = Net()
```
b

## Save/Load model

| | |
|---|---|
| **model = torch.load('PATH')** | Load model |
| **torch.save(model, 'PATH')** | Save model |

*It is common practice to save only the model parameters, not the whole model using model.state_dict()*

```
torch.save(model.state_dict(), 'params.ckpt')
model.load_state_dict(
                  torch.load('params.ckpt'))
```

## GPU Training

**device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')**

If a GPU with CUDA support is available, computations are sent to the GPU with ID 0 using **model.to(device)** or **inputs, labels = data[0].to(device), data[1].to(device)**.

## Train model

### LOSS FUNCTIONS

PyTorch already offers a bunch of different loss fuctions, e.g.:

| | |
|---|---|
| **nn.L1Loss** | Mean absolute error |
| **nn.MSELoss** | Mean squared error (L2Loss) |
| **nn.CrossEntropyLoss** | Cross entropy, e.g. for single-label classification or unbalanced training set |
| **nn.BCELoss** | Binary cross entropy, e.g. for multi-label classification or autoencoders |

### OPTIMIZATION (torch.optim)

Optimization algorithms are used to update weights and dynamically adapt the learning rate with gradient descent, e.g.:

| | |
|---|---|
| **optim.SGD** | Stochastic gradient descent |
| **optim.Adam** | Adaptive moment estimation |
| **optim.Adagrad** | Adaptive gradient |
| **optim.RMSProp** | Root mean square prop |

```
import torch.optim as optim

# Define loss function
loss_fn = nn.CrossEntropyLoss()

# Choose optimization method
optimizer = optim.SGD(model.parameters(),
                      lr=0.001, momentum=0.9)

# Loop over dataset multiple times (epochs)
for epoch in range(2):
    model.train() # activate training mode
    for i, data in enumerate(train_loader, 0):
        # data is a batch of [inputs, labels]
        inputs, labels = data

        # zero gradients
        optimizer.zero_grad()

        # calculate outputs
        outputs = model(inputs)
        # calculate loss & backpropagate error
        loss = loss_fn(outputs, labels)
        loss.backward()
        # update weights & learning rate
        optimizer.step()
```

```
correct = 0 # correctly classified
total   = 0 # classified in total

model.eval()
with torch.no_grad():
  for data in test_loader:
    inputs, labels = data
    outputs = model(inputs)
    _, predicted = torch.max(outputs.data, 1)
    total += labels.size(0) # batch size
    correct += (predicted==labels)
                          .sum().item()

print('Accuracy: %s' % (correct/total))
```

## Evaluate model

The evaluation examines whether the model provides satisfactory results on previously withheld data. Depending on the objective, different metrics are used, such as acurracy, precision, recall, F1, or BLEU.

| | |
|---|---|
| **model.eval()** | Activates evaluation mode, some layers behave differently |
| **torch.no_grad()** | Prevents tracking history, reduces memory usage, speeds up calculations |

Stefan Seegerer, hi@stefanseegerer.de • Matthias Zürl, matthias.zuerl@fau.de
• CC-BY-SA
• Last updated: 10/2021

```
    # SGD with Momentum (recommended)
    optimizer = optim.SGD(
        model.parameters(),    # Which parameters to optimize
        lr=0.01,               # Learning rate
        momentum=0.9           # Momentum factor (0.9 is
    common)
    )
    optimizer = optim.RMSprop(
        model.parameters(),
        lr=0.001,          # learning rate
        alpha=0.99,        # decay rate for squared gradients
        eps=1e-8,          # numerical stability
        weight_decay=0,    # L2 regularization
        momentum=0         # optional momentum
    )
 optimizer = optim.Adam(
     model.parameters(),
     lr=0.001,              # Learning rate (default: 0.001)
     betas=(0.9, 0.999),    # Coefficients for moving
 averages
     eps=1e-8,              # Numerical stability
     weight_decay=0         # L2 regularization
```

```
# MUST DO IN THIS ORDER:
optimizer.zero_grad()       # 1. Clear old
gradients
-----------------------
---------------------
loss.backward()             # 2. Compute new
gradients
```

```
        # Pooling
        self.gap = nn.AdaptiveAvgPool2d((1, 1)) #16 × 64 × 64  16 × 1
        × 1
        # Pooling
        self.pool = nn.MaxPool2d(2, 2)
        # [B, 16, 64, 64]  [B, 16, 32, 32]
```

```
    ## LOSS FUNCTIONS:
    criterion = nn.CrossEntropyLoss(). # for multi
    class
    criterion = nn.BCEWithLogitsLoss(). # binary
```

```
 # Setup
 model = SimpleCNN().to(device)
 criterion = nn.CrossEntropyLoss()
 optimizer = optim.Adam(model.parameters(),
```

```
# training

for images, labels in train_loader:
        # Move data to device
        images = images.to(device)   # [batch, 1,
64, 64]
        labels = labels.to(device)   # [batch]

        # Forward pass
        optimizer.zero_grad()       # Clear gradients
        outputs = model(images)      # [batch, 10]
        loss = criterion(outputs, labels)

        # Backward pass
        loss.backward()             # Compute
gradients
        optimizer.step()            # Update weights
```

```
# inference

model.eval()                    # Disable dropout, batchnorm in eval
mode
correct = 0
total = 0

with torch.no_grad():               # Disable gradient computation
(faster)
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        _, predicted = outputs.max(1)
```

```
    # Save model
    torch.save(model.state_dict(), 'model.pth')

    # Load model
    model = SimpleCNN()
    model.load_state_dict(torch.load
    ('model.pth'))
```

```
    ========================================
    =
    # data loading and transformation
    from torchvision import datasets, transforms
    from torch.utils.data import DataLoader
```

```
transform = transforms.Compose([
    transforms.Resize((64, 64)),      # Resize all images
    transforms.ToTensor(),            # Convert to tensor
[0,1]
    transforms.Normalize(             # Normalize RGB
images
        mean=(0.5, 0.5, 0.5),
        std=(0.5, 0.5, 0.5)
    )
])

 # use Imagefolder ( auto returns images and labels)

train_dataset = datasets.ImageFolder(
    root="./dataset/train",   # path to training images
    transform=transform
)

test_dataset = datasets.ImageFolder(
    root="./dataset/test",    # path to testing images
    transform=transform
)

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32,
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "
cpu")
```

```
            nn.Conv2d(
                in_channels=3,
                out_channels=32,
                kernel_size=3,
                stride=1,
                padding=0   #
            default
            )
```

```
import torch
import torch.nn as nn

class CNN(nn.Module):
    def __init__(self, num_classes=10):
        super(CNN, self).__init__()

        # Feature extractor (Conv blocks)
        self.features = nn.Sequential(
            # Block 1
            nn.Conv2d(1, 32, kernel_size=3, padding=1),  # [B,1,28,28]
[B,32,28,28]
            nn.BatchNorm2d(32),  # output channel r count ta e boshabo
            nn.ReLU(),
            nn.MaxPool2d(2, 2),         (kernel,stride)                #  [B,32,14,14]

            # Block 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1),  #  [B,64,14,14]
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)                          #  [B,64,7,7]
        )

        # Classifier (FC layers)
        self.classifier = nn.Sequential(
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, num_classes)
        )
```

```
    def forward(self, x):
        x = self.features(x)            # [B,64,7,7]
        x = x.view(x.size(0), -1)         # [B,3136]
        x = self.classifier(x)          # [B,10]
        return x


model = Net().to(device)

# ===== 4. LOSS & OPTIMIZER =====
criterion = nn.CrossEntropyLoss()
```

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()

        self.conv1 = nn.Conv2d(
            in_channels=1, #MNIST is grayscale
            out_channels=16, #16 feature maps
            kernel_size=3,
            stride=1,
            padding=1
        )
        # input : 1 × 64 × 64
        #output: 16 × 64 × 64

        self.relu = nn.ReLU()


        self.gap = nn.AdaptiveAvgPool2d((1, 1)) #16 × 64 × 64  16 × 1
× 1

        self.fc = nn.Linear(16, 10) #Input: 16 numbers (from GAP)
```